

Laboratório 05

Parte 2

Neste laboratório o objetivo é implementar um microcontrolador simples de 16 bits. O laboratório está dividido em 5 partes, e neste documento iremos detalhar apenas a segunda destas.

Parte 2

Nesta parte você deve implementar o circuito da Figura 4. Observe que o processador usado é exatamente o mesmo do projetado na parte 1, então não devem aparecer dificuldades maiores neste ponto caso da parte 1 esteja funcionando corretamente. Devemos apenas implementar, exteriormente ao processador, um contador (que servirá de apontador de uma memória ROM) e esta memória propriamente dita, que servirá para armazenar as instruções (e dados quando seguirem uma instrução mvi). Observe também que temos dois sinais de clock, um do conjunto contador/memória e outro do processador. Nesta parte, o sincronismo destes deverá ser feito manualmente em simulação. Este ponto será alterado na parte 3.

A criação do componente de memória ROM exige um arquivo que indique o seu conteúdo. Existem duas formas de definir estes dados, através de um arquivo .mif (memory initialization file) e .hex (hexadecimal file). O 1º define o conteúdo a partir de um arquivo texto, sendo mais fácil de ser lido. Por exemplo, para implementar o mesmo programa utilizado no teste da parte 1, o arquivo .mif será:

```
DEPTH = 128;
WIDTH = 16;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;

CONTENT BEGIN

00 : 0010000000000000;          %      mvi R0,#1      %
01 : 0000000000000001;
02 : 0000010000000000;          %      mv R1,R0      %
03 : 0100010000000000;          %      add R1,R0      %
04 : 0100010000000000;          %      add R1,R0      %
05 : 0110010000000000;          %      sub R1,R0      %

END;
```

Observe que as primeiras linhas definem o número máximo de palavras (depth), o número de bits por palavra (width), e as bases numéricas usadas para representar o endereço e o dado, respectivamente. O segundo bloco, de dados, descreve linha a linha o conteúdo do endereço em questão. Comentários podem ser colocados entre (%...%).

Infelizmente, o ModelSim não é capaz de simular memórias inicializadas com arquivos deste tipo. Por isso, é necessário que, uma vez criado, este arquivo seja convertido em .hex através do

Quartus, e que este novo arquivo seja indicado como quem inicializa a memória durante a criação do componente. **Importante:** não se esqueça de retirar o registrador de saída da memória durante sua configuração.

Novamente incluímos a diretriz

```
attribute keep : boolean;  
attribute keep of DIN: signal is true;
```

para poder visualizar a saída da memória em simulação.

Para questão de simulação, iremos implementar apenas os componentes sem conectar aos periféricos da placa. Desta forma, a entidade a ser usada é:

```
ENTITY lab12part2 IS  
    PORT ( MClock,PClock,Resetn,Run      : IN STD_LOGIC;  
          BusWires                        : BUFFER STD_LOGIC_VECTOR(15 DOWNT0 0);  
          Done                            : BUFFER STD_LOGIC);  
END lab12part2;
```

O resultado da simulação obtida foi:

